

ECGR 4105 Midterm Project 1

Group members: Jordan Mckoy, Miguel Diaz-Alvarez, AJ Carroll

Import Libraries and Load Data

We import numpy, pandas, and matplotlib for data handling and plotting, plus everything we need from scikit-learn: the dataset, the six classifiers, the scalers, and the tools for splitting, cross-validation, grid search, and scoring.

```
In [13]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    make_scorer,
    precision_score,
    recall_score,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression, Perceptron
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
import jinja2
```

Data Preprocessing

We load the breast cancer dataset and split it 80/20 into training and test sets. We use `stratify=y` so malignant and benign cases show up in the same proportions in both sets as in the full dataset. We do NOT scale the data here — scaling happens inside each model's Pipeline so the scaler only ever learns from training data. This prevents data leakage.

```
In [14]: RANDOM_STATE = 42
data = load_breast_cancer()
X, y = data.data, data.target
display(pd.DataFrame(X, columns=data.feature_names).head())
display(pd.DataFrame(y, columns=["target"]).head())
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)
# scaler = StandardScaler()
# X_train_scaled = scaler.fit_transform(X_train)
# X_test_scaled = scaler.transform(X_test)

```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	s
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	

5 rows × 30 columns

	target
0	0
1	0
2	0
3	0
4	0

Evaluate Model Helper

This function does the same evaluation for every model so we don't repeat code. It trains the model on the training set, predicts on the test set, plots the confusion matrix, prints the classification report, and returns the metrics (accuracy, precision, recall, F1, and malignant recall) as a dictionary so we can build a comparison table later.

```

In [15]: def evaluate(model, name):
          model.fit(X_train, y_train)
          y_pred = model.predict(X_test)
          cm = confusion_matrix(y_test, y_pred)
          ConfusionMatrixDisplay(cm, display_labels=data.target_names).plot(cmap=p
          plt.title(name)
          plt.show()

```

```

print(classification_report(y_test, y_pred, target_names=data.target_names))
return {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred),
    "recall": recall_score(y_test, y_pred),
    "f1_score": f1_score(y_test, y_pred),
    "recall_malignant": recall_score(y_test, y_pred, pos_label=0),
}

```

Grid Search Summarizer

This function prints the best parameters and best cross-validation score found by each grid search. If the search included different scalers, it also shows a small table comparing how each scaler performed, which we use to answer the discussion question about how scaling affects each algorithm.

```

In [16]: def summarize_grid(grid, name):
    print(f"{name} - best params: {grid.best_params_}")
    print(f"{name} - best CV score: {grid.best_score_:.4f}")
    cv = pd.DataFrame(grid.cv_results_)
    if "param_scaler" in cv.columns:
        summary = (cv.groupby(cv["param_scaler"].astype(str))["mean_test_score"]
                    .agg(["max", "mean"]).round(4)
                    .sort_values("max", ascending=False))
        display(summary)
    return cv

```

Custom Scorer: Weighted Recall

This scorer computes $0.6 \times \text{malignant recall} + 0.4 \times \text{benign recall}$. In a medical context, a false negative (missing a cancer case) is worse than a false positive, so we weight malignant recall more heavily. We don't use 100% malignant recall because a model could cheat by predicting everyone has cancer. Grid search uses this scorer to pick the best parameters.

```

In [17]: def weighted_recall(y_true, y_pred, w_m = 0.6, w_b = 0.4):
    recall_m = recall_score(y_true, y_pred, pos_label=0)
    recall_b = recall_score(y_true, y_pred, pos_label=1)
    return (w_m * recall_m) + (w_b * recall_b)

```

Model Training Approach

Every classifier below follows the same pattern, so we explain it once here.

Pipeline: Each model is wrapped in a Pipeline with two steps: a scaler and the classifier. Putting the scaler inside the pipeline means it is re-fit on only the training portion of each cross-validation fold, so no information from the validation data ever leaks into the scaling. The Decision Tree is the one exception — it has no scaler, since

trees split on thresholds and rescaling a feature cannot change any split.

GridSearchCV: We define a grid of parameter values for each model and GridSearchCV tries every combination, scoring each one with 5-fold stratified cross-validation on the training set. Stratified folds keep the malignant/benign ratio the same in every fold. Each grid also includes the scaler itself as a parameter (StandardScaler, MinMaxScaler, or none), so the search picks the best scaler per model instead of us guessing. All searches use our weighted recall scorer to select the winner.

Choosing which parameters to test: For each classifier we read the scikit-learn documentation to find the parameters that most affect the model's behavior (for example C and solver for Logistic Regression, kernel/C/gamma for SVM, max_depth and pruning options for the Decision Tree, n_neighbors and weights for KNN) and picked value ranges around the defaults. After the search, `summarize_grid` prints the best parameters and compares how each scaler did.

The best estimator from each search is then passed to `evaluate`, which retrains it on the full training set and reports its confusion matrix and metrics on the held-out test set.

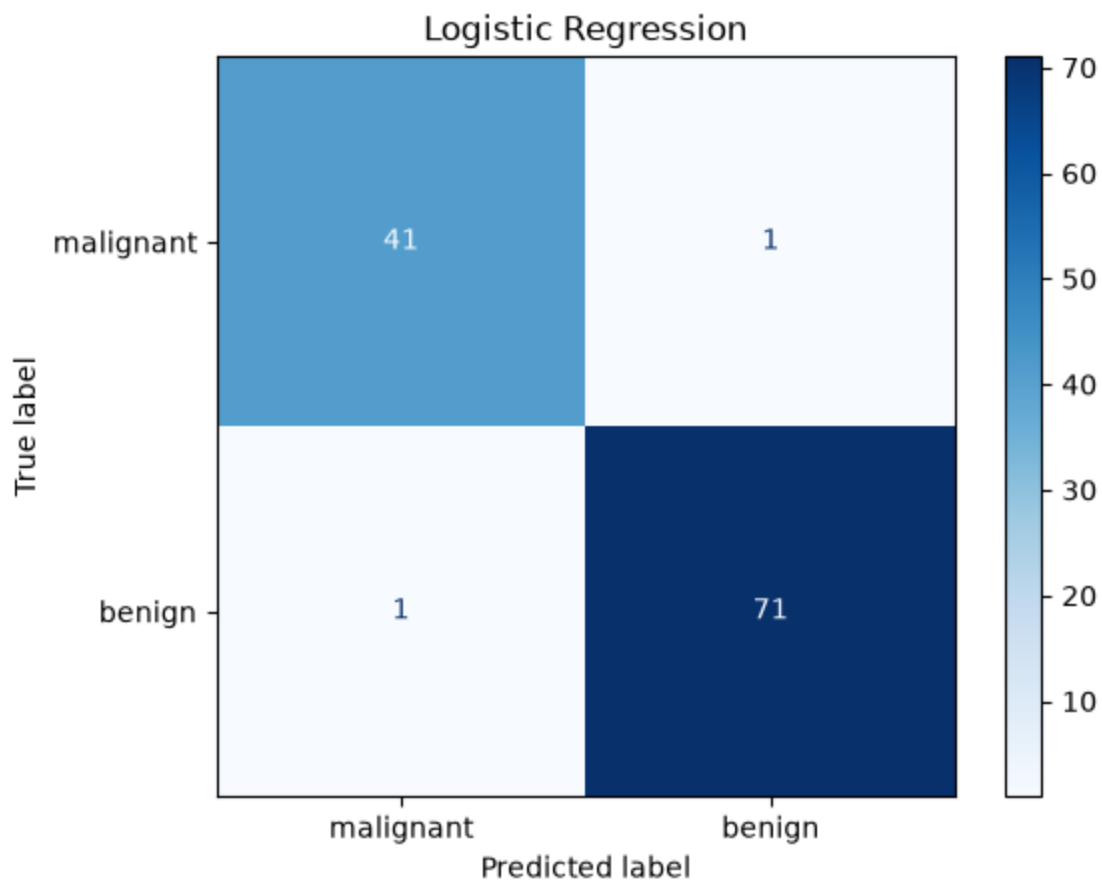
Logistic Regression

```
In [18]: log_grid = GridSearchCV(
    Pipeline([("scaler", StandardScaler()), ("clf", LogisticRegression(random_state=0,
    {
        "clf__C": [0.01, 0.1, 1, 10, 100],
        "clf__solver": ["liblinear", "lbfgs"],
        "clf__max_iter": [2000, 5000],
        "scaler": [StandardScaler(), MinMaxScaler()],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    # scoring=make_scorer(recall_score, pos_label=0),
    scoring=make_scorer(weighted_recall)
    )
    log_grid.fit(X_train, y_train)
    log_cv = summarize_grid(log_grid, "Logistic Regression")
    log_metrics = evaluate(log_grid.best_estimator_, "Logistic Regression")
```

Logistic Regression – best params: {'clf__C': 1, 'clf__max_iter': 2000, 'clf__solver': 'liblinear', 'scaler': StandardScaler()}

Logistic Regression – best CV score: 0.9767

	max	mean
param_scaler		
StandardScaler()	0.9767	0.9599
MinMaxScaler()	0.9746	0.8970



	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

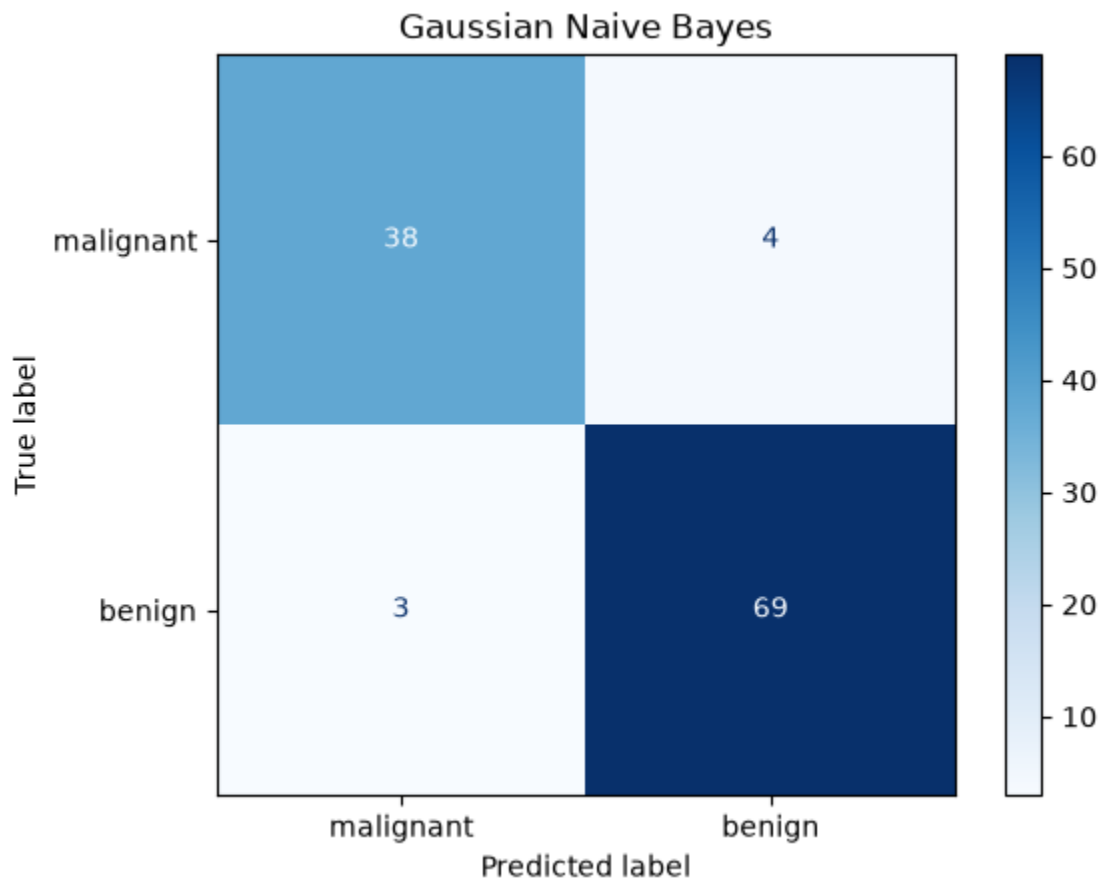
Naive Bayes

```
In [19]: nb_grid = GridSearchCV(Pipeline([("scaler", StandardScaler()), ("clf", Gauss
    {
        "scaler": [StandardScaler(), MinMaxScaler(), "passthrough"],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    scoring=make_scorer(weighted_recall),
    )
nb_grid.fit(X_train,y_train)
nb_cv = summarize_grid(nb_grid, "Naive Bayes")
naive_metrics = evaluate(nb_grid.best_estimator_, "Gaussian Naive Bayes")
```

Naive Bayes – best params: {'scaler': 'passthrough'}

Naive Bayes – best CV score: 0.9260

	max	mean
param_scaler		
passthrough	0.9260	0.9260
MinMaxScaler()	0.9218	0.9218
StandardScaler()	0.9218	0.9218



	precision	recall	f1-score	support
malignant	0.93	0.90	0.92	42
benign	0.95	0.96	0.95	72
accuracy			0.94	114
macro avg	0.94	0.93	0.93	114
weighted avg	0.94	0.94	0.94	114

Perceptron

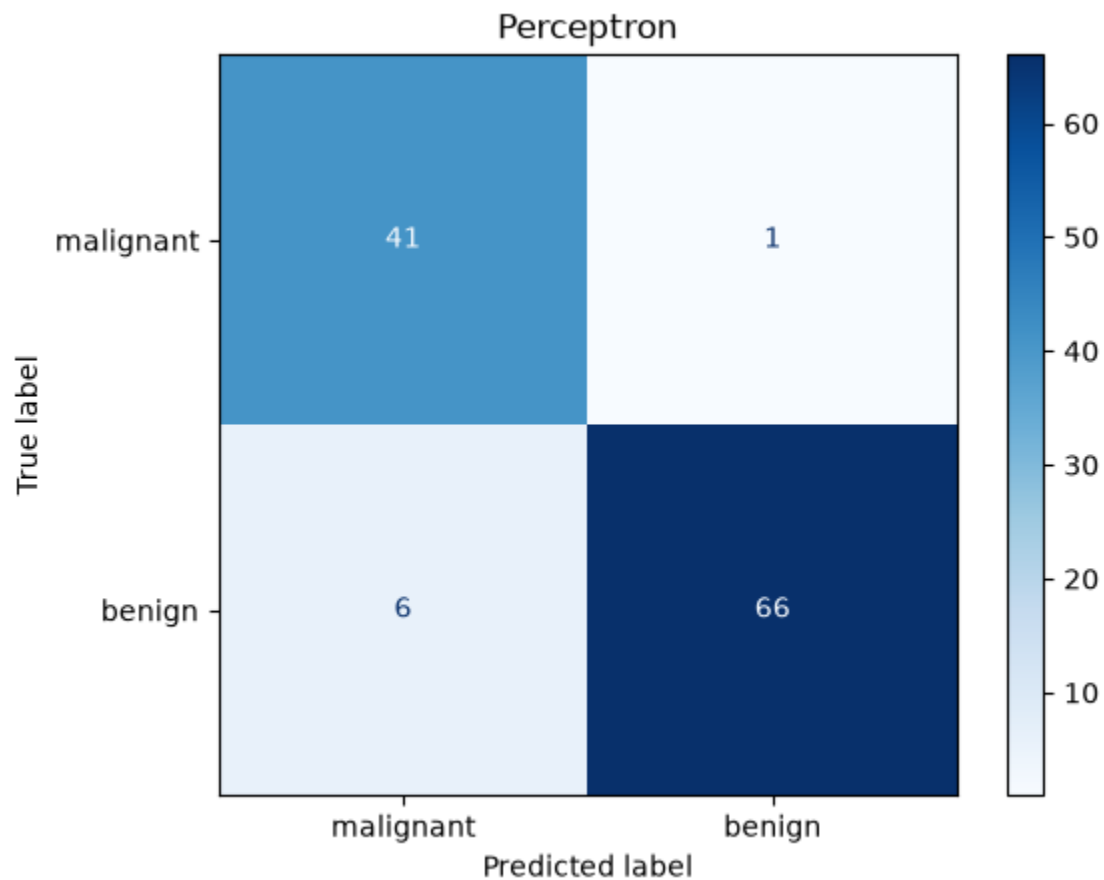
```
In [20]: per_grid = GridSearchCV(
    Pipeline([("scaler", StandardScaler()), ("clf", Perceptron(random_state=
    {
        "clf__penalty": [None, "l2", "l1", "elasticnet"],
        "clf__alpha": [0.0001, 0.001, 0.01],
        "clf__max_iter": [1000, 2000],
        "scaler": [StandardScaler(), MinMaxScaler(), "passthrough"],
```

```
},
cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
# scoring=make_scorer(recall_score, pos_label=0),
scoring=make_scorer(weighted_recall)
)
per_grid.fit(X_train, y_train)
per_cv = summarize_grid(per_grid, "Perceptron")
per_metrics = evaluate(per_grid.best_estimator_, "Perceptron")
```

Perceptron – best params: {'clf__alpha': 0.0001, 'clf__max_iter': 1000, 'clf__penalty': 'elasticnet', 'scaler': StandardScaler()}

Perceptron – best CV score: 0.9577

	max	mean
param_scaler		
StandardScaler()	0.9577	0.9373
MinMaxScaler()	0.9514	0.9074
passthrough	0.8669	0.7718



	precision	recall	f1-score	support
malignant	0.87	0.98	0.92	42
benign	0.99	0.92	0.95	72
accuracy			0.94	114
macro avg	0.93	0.95	0.94	114
weighted avg	0.94	0.94	0.94	114

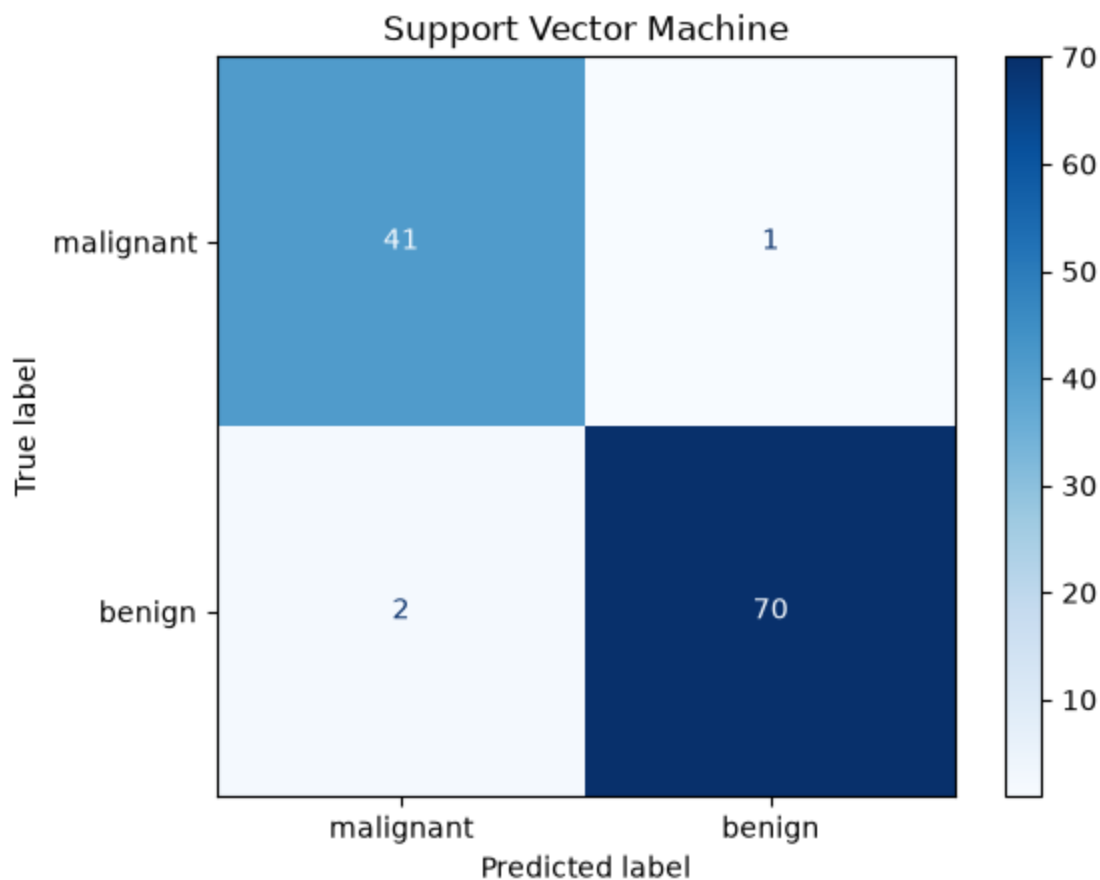
Support Vector Machine, SVM

```
In [21]: svm_grid = GridSearchCV(
    Pipeline([("scaler", StandardScaler()), ("clf", SVC())]),
    {
        "clf__kernel": ["linear", "rbf"],
        "clf__C": [0.1, 1, 10, 100],
        "clf__gamma": ["scale", "auto"],
        "scaler": [StandardScaler(), MinMaxScaler(), "passthrough"],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    # scoring=make_scorer(recall_score, pos_label=0),
    scoring=make_scorer(weighted_recall)
)
svm_grid.fit(X_train, y_train)
svm_cv = summarize_grid(svm_grid, "SVM")
svm_metrics = evaluate(svm_grid.best_estimator_, "Support Vector Machine")
```

SVM – best params: {'clf__C': 10, 'clf__gamma': 'scale', 'clf__kernel': 'rbf', 'scaler': MinMaxScaler()}

SVM – best CV score: 0.9711

	max	mean
param_scaler		
MinMaxScaler()	0.9711	0.9316
StandardScaler()	0.9662	0.9551
passthrough	0.9521	0.7900



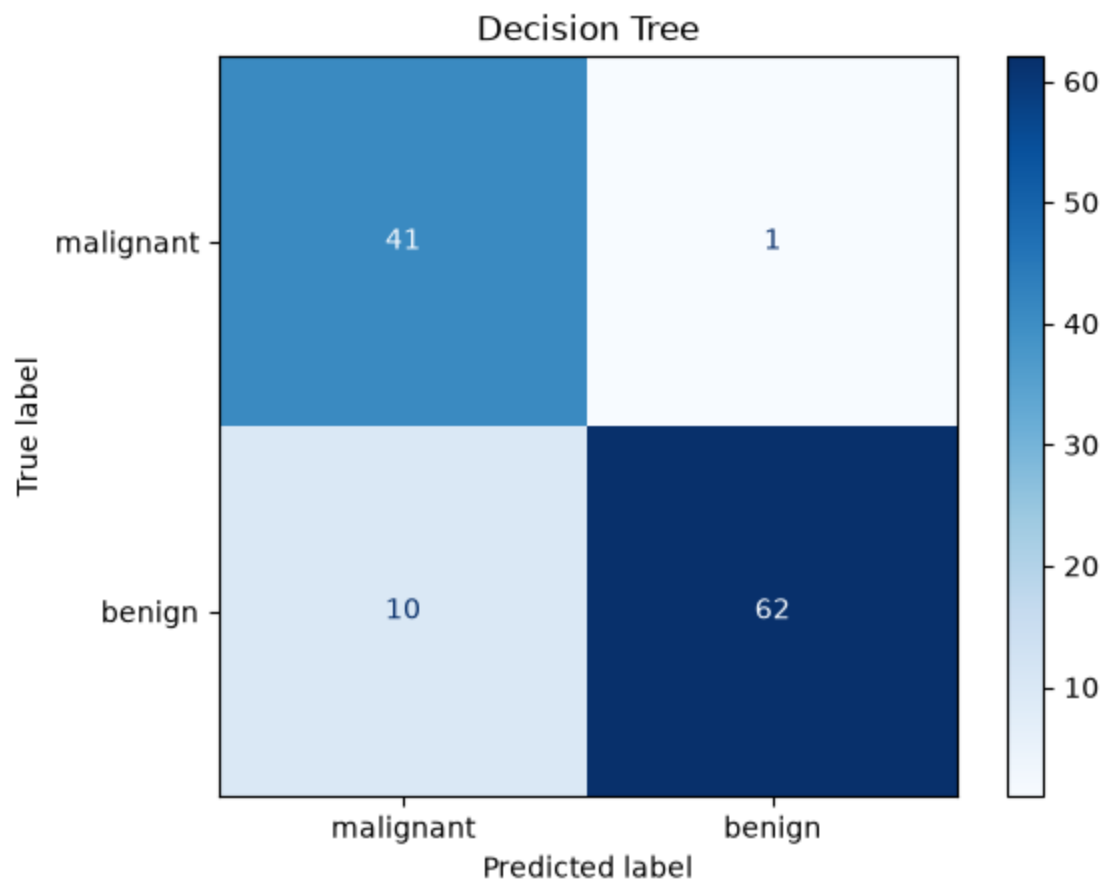
	precision	recall	f1-score	support
malignant	0.95	0.98	0.96	42
benign	0.99	0.97	0.98	72
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Decision Tree Classifier

```
In [22]: dt_grid = GridSearchCV(
    Pipeline([("clf", DecisionTreeClassifier(random_state=RANDOM_STATE))]),
    {
        "clf__criterion": ["gini", "entropy"],
        "clf__max_depth": [None, 3, 5, 10],
        "clf__ccp_alpha": [0.0, 0.001, 0.01],
        "clf__min_samples_leaf": [1, 5, 10, 20],
        "clf__class_weight": [None, "balanced"],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    # scoring=make_scorer(recall_score, pos_label=0),
    scoring=make_scorer(weighted_recall)
)
dt_grid.fit(X_train, y_train)
dt_cv = summarize_grid(dt_grid, "Decision Tree")
dt_metrics = evaluate(dt_grid.best_estimator_, "Decision Tree")
```

Decision Tree – best params: {'clf__ccp_alpha': 0.0, 'clf__class_weight': None, 'clf__criterion': 'gini', 'clf__max_depth': None, 'clf__min_samples_leaf': 20}

Decision Tree – best CV score: 0.9374



	precision	recall	f1-score	support
malignant	0.80	0.98	0.88	42
benign	0.98	0.86	0.92	72
accuracy			0.90	114
macro avg	0.89	0.92	0.90	114
weighted avg	0.92	0.90	0.90	114

K-Nearest Neighbors, KNN

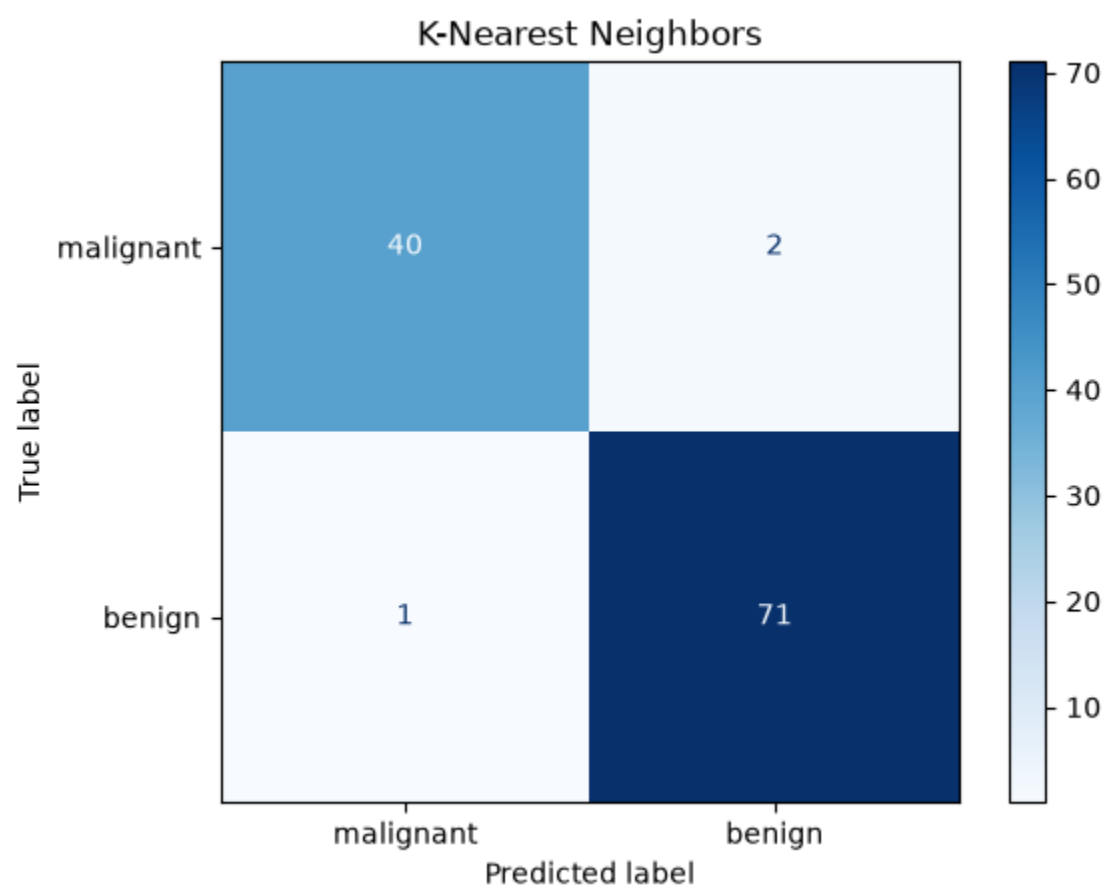
```
In [23]: knn_grid = GridSearchCV(
    Pipeline([("scaler", StandardScaler()), ("clf", KNeighborsClassifier())]
    {
        "clf__n_neighbors": [3, 5, 7, 9, 11, 13],
        "clf__weights": ["uniform", "distance"],
        "scaler": [StandardScaler(), MinMaxScaler(), "passthrough"],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    # scoring=make_scorer(recall_score, pos_label=0),
    scoring=make_scorer(weighted_recall)
)
knn_grid.fit(X_train, y_train)
```

```
knn_cv = summarize_grid(knn_grid, "KNN")
knn_metrics = evaluate(knn_grid.best_estimator_, "K-Nearest Neighbors")
```

KNN – best params: {'clf__n_neighbors': 7, 'clf__weights': 'uniform', 'scaler': MinMaxScaler() }

KNN – best CV score: 0.9662

	max	mean
param_scaler		
MinMaxScaler()	0.9662	0.9614
StandardScaler()	0.9520	0.9487
passthrough	0.9281	0.9174



	precision	recall	f1-score	support
malignant	0.98	0.95	0.96	42
benign	0.97	0.99	0.98	72
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Comparison of Models

We collect the test-set metrics from the best version of each classifier into one table,

sort by weighted recall, and plot everything on a bar chart. We also print the table in LaTeX format for the report. Finally, we run 10-fold cross-validation on each best model over the training data to see the mean and standard deviation of the weighted recall — this shows how stable each model's performance is, not just its single test-set score.

```
In [24]: from sklearn.model_selection import cross_val_score
results = pd.DataFrame({
    "Logistic Regression": log_metrics,
    "Naive Bayes": naive_metrics,
    "Perceptron": per_metrics,
    "SVM": svm_metrics,
    "Decision Tree": dt_metrics,
    "KNN": knn_metrics,
}).T

# display(results.round(4))
results["weighted_recall"] = 0.6 * results["recall_malignant"] + 0.4 * results["recall_benign"]
final = results.sort_values("weighted_recall", ascending=False).round(4)
display(final)

latex_table = (
    final.rename(columns={
        "accuracy": "Accuracy",
        "precision": "Precision",
        "recall": "Recall",
        "f1_score": "F1",
        "recall_malignant": "Recall (Mal.)",
        "weighted_recall": "Weighted Recall",
    })
    .style.format(precision=4)
    .to_latex(hrules=True, column_format="lcccccc")
)
print(latex_table)

ax = results.plot(kind="bar", figsize=(11, 5), rot=20, width=0.8,
                  edgecolor="black", linewidth=0.5)
ax.set_ylim(0.85, 1.0)
ax.set_ylabel("Score")
ax.set_title("Classifier Performance Comparison (Validation Set)")
ax.legend(loc="lower right")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

models = {
    "Logistic Regression": log_grid.best_estimator_,
    "Naive Bayes": nb_grid.best_estimator_,
    "Perceptron": per_grid.best_estimator_,
    "SVM": svm_grid.best_estimator_,
    "Decision Tree": dt_grid.best_estimator_,
    "KNN": knn_grid.best_estimator_,
}
```

```

cv10 = StratifiedKFold(10, shuffle=True, random_state=RANDOM_STATE)
scorer = make_scorer(weighted_recall)

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=cv10, scoring=score
    print(f"{name:<22} {scores.mean():.4f} ± {scores.std():.4f}")

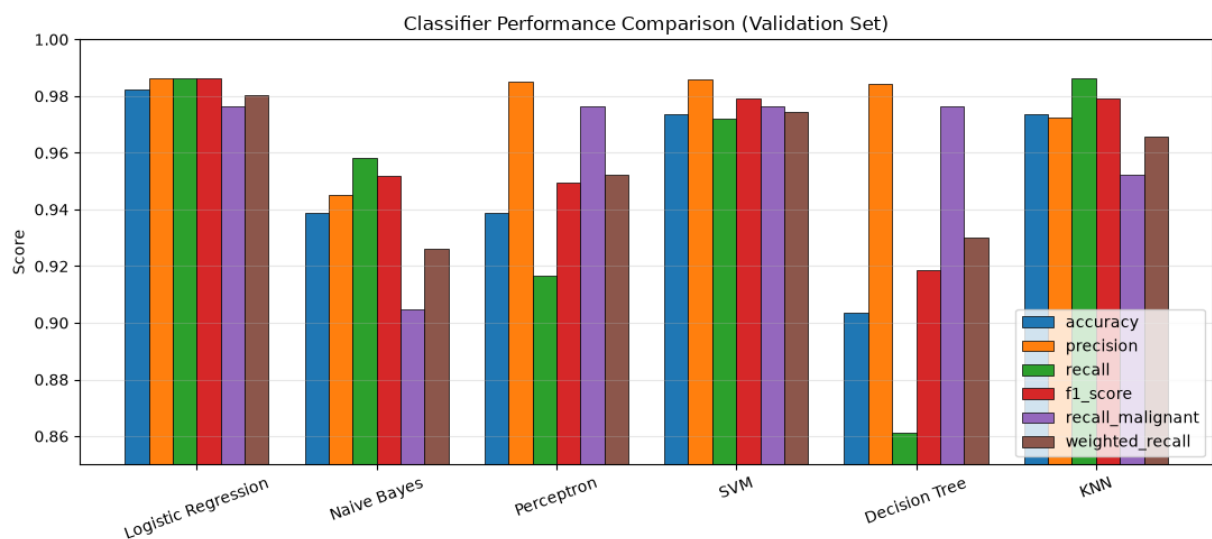
```

	accuracy	precision	recall	f1_score	recall_malignant	weighted_recall
Logistic Regression	0.9825	0.9861	0.9861	0.9861	0.9762	0.9802
SVM	0.9737	0.9859	0.9722	0.9790	0.9762	0.9746
KNN	0.9737	0.9726	0.9861	0.9793	0.9524	0.9659
Perceptron	0.9386	0.9851	0.9167	0.9496	0.9762	0.9524
Decision Tree	0.9035	0.9841	0.8611	0.9185	0.9762	0.9302
Naive Bayes	0.9386	0.9452	0.9583	0.9517	0.9048	0.9262

```

\begin{tabular}{lcccccc}
\toprule
& Accuracy & Precision & Recall & F1 & Recall (Mal.) & Weighted Recall \\
\midrule
Logistic Regression & 0.9825 & 0.9861 & 0.9861 & 0.9861 & 0.9762 & 0.9802 \\
SVM & 0.9737 & 0.9859 & 0.9722 & 0.9790 & 0.9762 & 0.9746 \\
KNN & 0.9737 & 0.9726 & 0.9861 & 0.9793 & 0.9524 & 0.9659 \\
Perceptron & 0.9386 & 0.9851 & 0.9167 & 0.9496 & 0.9762 & 0.9524 \\
Decision Tree & 0.9035 & 0.9841 & 0.8611 & 0.9185 & 0.9762 & 0.9302 \\
Naive Bayes & 0.9386 & 0.9452 & 0.9583 & 0.9517 & 0.9048 & 0.9262 \\
\bottomrule
\end{tabular}

```



Logistic Regression	0.9781 ± 0.0156
Naive Bayes	0.9246 ± 0.0553
Perceptron	0.9619 ± 0.0423
SVM	0.9676 ± 0.0187
Decision Tree	0.9211 ± 0.0537
KNN	0.9675 ± 0.0282